

軟體工程

第一課：軟體工程導論

授課教授：薛念林 教授 (與 Gemini AI 協作)

資訊工程學系

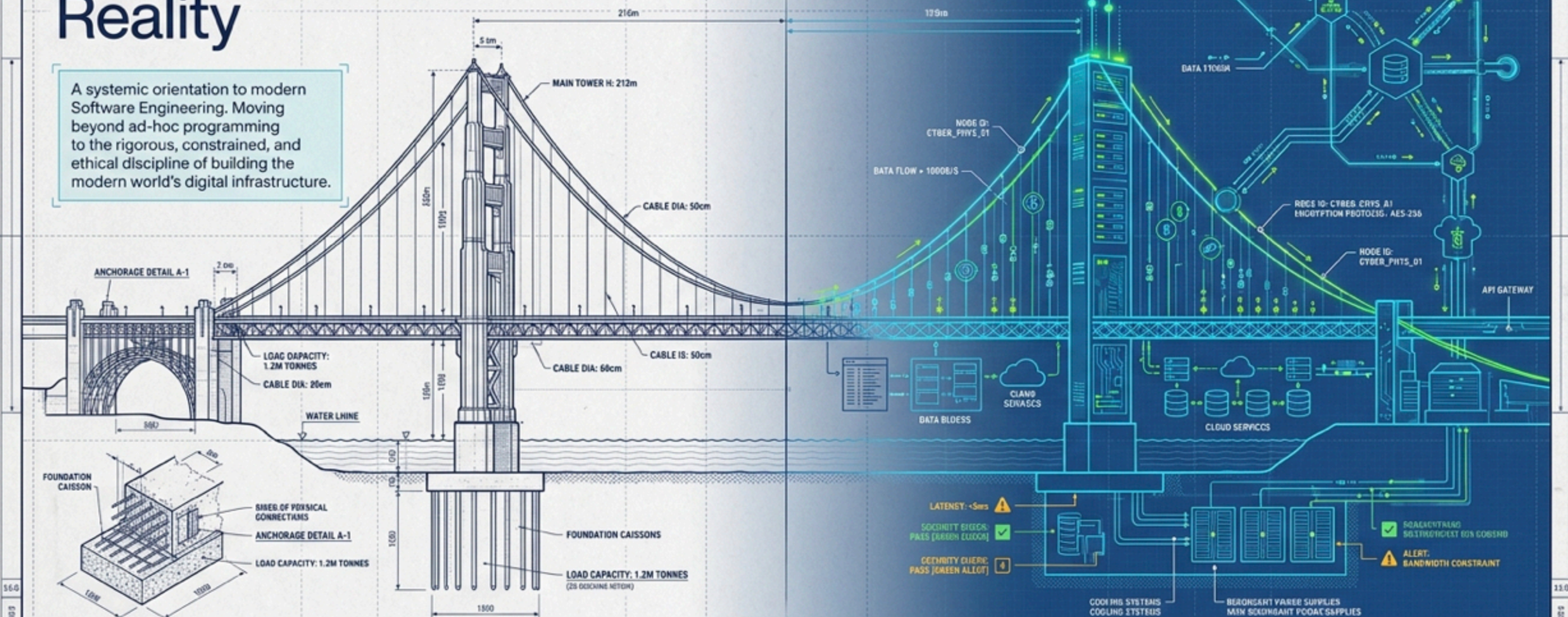
逢甲大學

第一章：課程大綱與核心概念

- **1.1 技術演進脈絡：** 從機械化到普及化 AI
- **1.2 軟體工程起源與「軟體危機」：** 1968 年 NATO 會議與重大歷史災難
- **1.3 軟體本質剖析：** 超越原始碼的四大支柱（程式、資料、程序、文檔）
- **1.4 ISO 9126 品質模型：** 6 大特徵與關鍵子屬性實務案例
- **1.5 現代軟體多元樣貌：** Web、Mobile、ERP、嵌入式 IoT、AI 平台
- **1.6 何謂軟體工程？：** 定義、流程、約束權衡天平、四大核心活動與迷思
- **1.7 現代工具鏈與 AI 雙面刃：** CI/CD 自動化與 AI 效益/風險矩陣
- **1.8 專業倫理與黑暗模式：** ACM/IEEE 倫理守則與 UI 欺騙模式
- **1.9 FAQ 與觀念檢測**

From Code to Cyber-Physical Reality

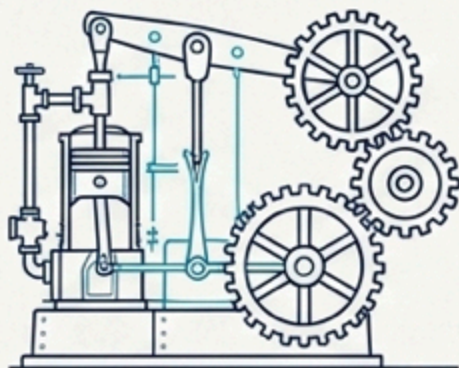
A systemic orientation to modern Software Engineering. Moving beyond ad-hoc programming to the rigorous, constrained, and ethical discipline of building the modern world's digital infrastructure.



The Evolution of Industrial Complexity

Industry 1.0 (Mechanization)

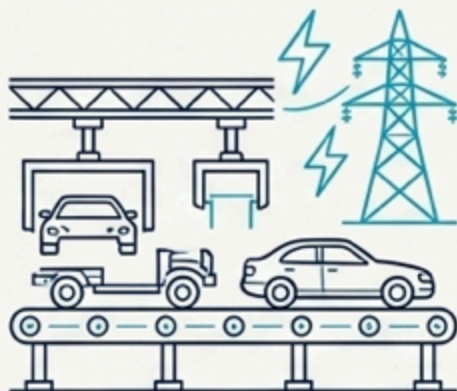
Late 18th-Mid 19th Century



Steam and water power.

Industry 2.0 (Mass Production)

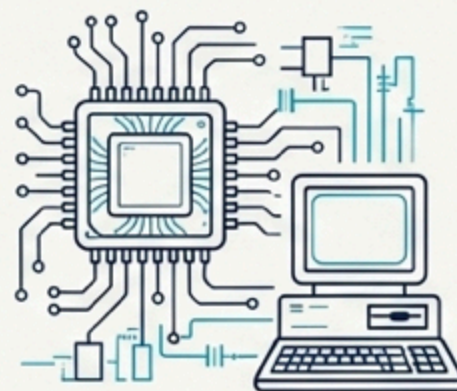
Late 19th-Early 20th Century



Electricity and assembly lines.

Industry 3.0 (Digital Revolution)

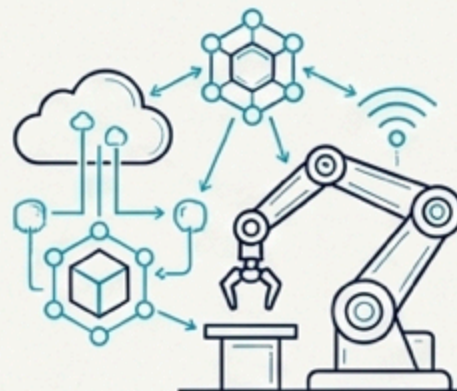
Mid-Late 20th Century



Computers, semiconductors, and PLCs.

Industry 4.0 (Cyber-Physical)

Present



IoT, AI, Cloud Computing, Advanced Robotics

Role of Software

1.0: Non-existent

2.0: Absent
(electro-mechanical)

3.0: Emergent
(Dawn of Software Engineering)

4.0: Pervasive
(Intelligent backbone)

1.1 四大工業革命時代

- **工業 1.0 — 機械化時代（18 世紀末至 19 世紀中葉）：**
 - 蒸汽與水力動力取代人體肌肉；工廠制度應運而生。
 - **軟體角色：** 無。純物理機械範疇。
- **工業 2.0 — 大規模量產時代（19 世紀末至 20 世紀初）：**
 - 電力、流水線裝配作業，推動標準化大規模生產。
 - **軟體角色：** 無。控制邏輯由物理電路硬焊決定。
- **工業 3.0 — 數位革命時代（20 世紀中葉至末期）：**
 - 半導體、微處理器、PLC 與 PC 普及。
 - **軟體角色：** 獨立成為一門專業領域，用於控制硬體與處理資料。
- **工業 4.0 — 互聯、雲端與 AI 時代（21 世紀至今）：**
 - 虛實整合系統 (CPS)、物聯網 (IoT)、雲端與自適應 AI。
 - **軟體角色：** 支撐現代社會運轉的中樞神經與核心基礎建設。

1.1 隨堂討論：課堂互動投票與討論

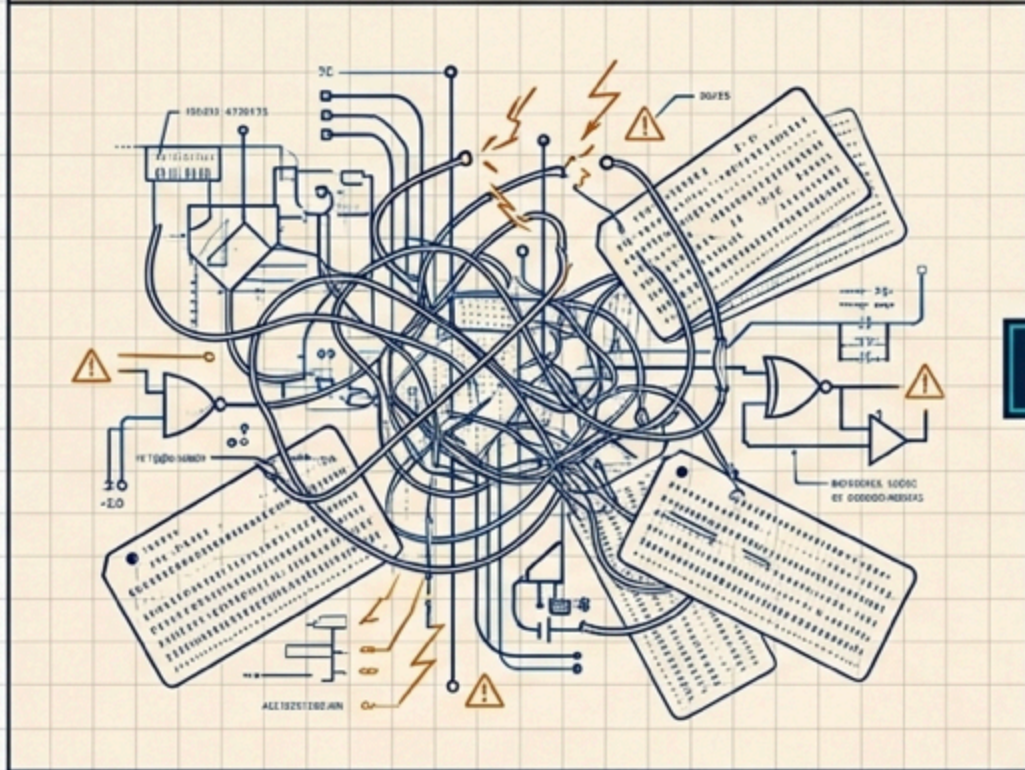
隨堂投票與小組討論：

- **投票：** 檢視你每天使用的各項數位服務與裝置，你認為其中有多少比例屬於工業 3.0 確定性邏輯，又有多少比例屬於工業 4.0 的自適應 AI 聯網軟體？
- **雙人討論：** 請列舉一個急需進行「工業 4.0 軟體轉型」的傳統流程（如校園停車、急診分流、公車排班）。在轉型過程中會面臨哪些獨特的軟體工程挑戰？



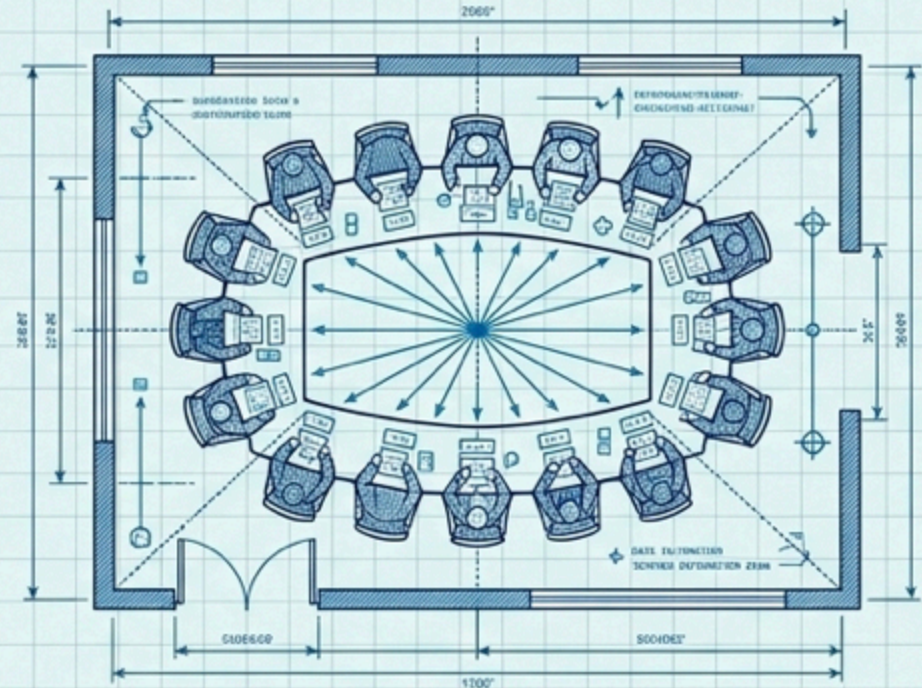
Surviving the Software Crisis

The Crisis (1960s/70s)



Rapidly scaling complexity exposed the flaws of custom, ad-hoc programming. Projects consistently suffered from massive budget overruns, late deliveries, and outright system failures.

The Genesis (1968 NATO Conferences)



Experts gathered to establish a formal discipline—borrowing the predictability, safety margins, and systematic planning of physical engineering to manage digital complexity.

1.2 1960 年代後期的軟體危機

- 隨著硬體成本急劇下降，軟體需求與複雜度呈爆炸性增長。
- 軟體危機的病徵：
 - 專案時程嚴重延宕，甚至完成前即遭廢棄。
 - 預算超支嚴重（常達最初預算的 3~4 倍）。
 - 系統品質低劣、頻繁崩潰且極難維護。
- 根本根源：將軟體開發視為孤立的手工藝做法，無法因應大型複雜系統。



1.2 1968 年 NATO Garmisch 會議

- 1968 年 10 月於德國加米施 (Garmisch) 召開：
 - 50 位頂尖電腦科學家、業界主管與學者匯聚。
 - 正式確立了 「軟體工程 (Software Engineering)」 一詞。
- 歷史使命：
 - 引導軟體開發從隨意無序的手工編程，轉型為具備嚴謹紀律的工程學科。
 - 引進結構化方法論、形式化規格、成本估算、嚴格驗證與專案管理。

1.2 軟體工程失敗的真實代價

- 名古屋空難 (1994) :

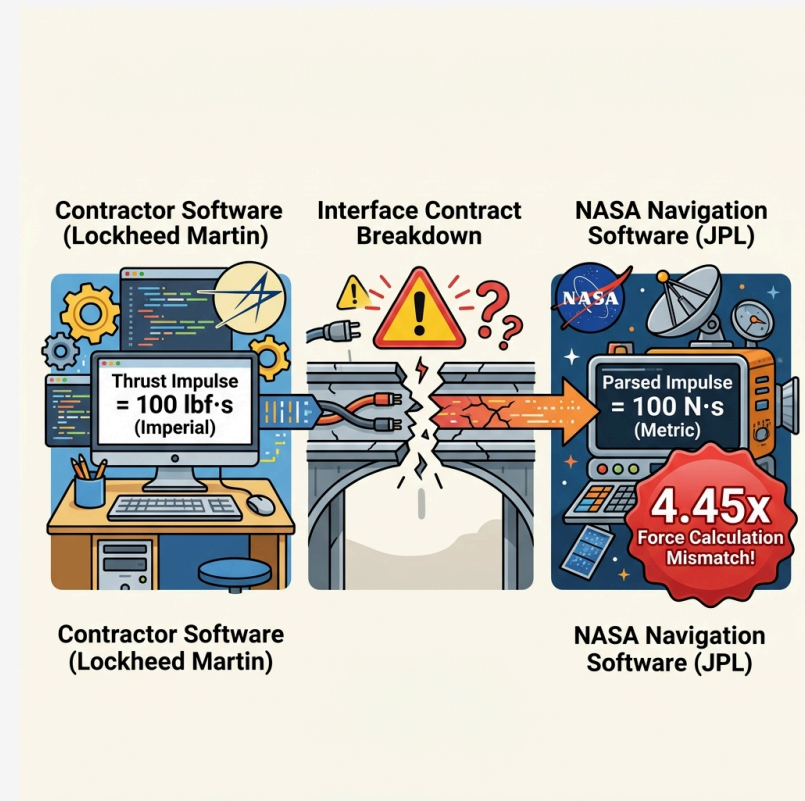
- 飛機自動駕駛重飛模式與飛行員手動操作產生軟體邏輯衝突，導致水平安定面被 trimmed 至極限，失速墜毀釀 264 死。

- 火星氣候探測器 (1999) :

- 地面軟體輸出英制單位 $\text{lbf} \cdot \text{s}$ ，太空船導航電腦預期公制單位 $\text{N} \cdot \text{s}$ ，因介面單位未驗證導致 3.27 億美元探測器毀滅。

- 阿利安 5 號火箭 501 航班 (1996) :

- 64 位元浮點數轉 16 位元有號整數發生溢位例外，缺乏處理程序導致電腦雙雙當機，發射 37 秒後自行引爆，損失 3.7 億美元。



觀念檢驗：軟體工程會議的召開 (CCQ 1)

第一次軟體工程會議在何時召開？

- **A.** 1928
- **B.** 1968
- **C.** 1948
- **D.** 1988



觀念檢驗：軟體危機的本質 (CCQ 2)

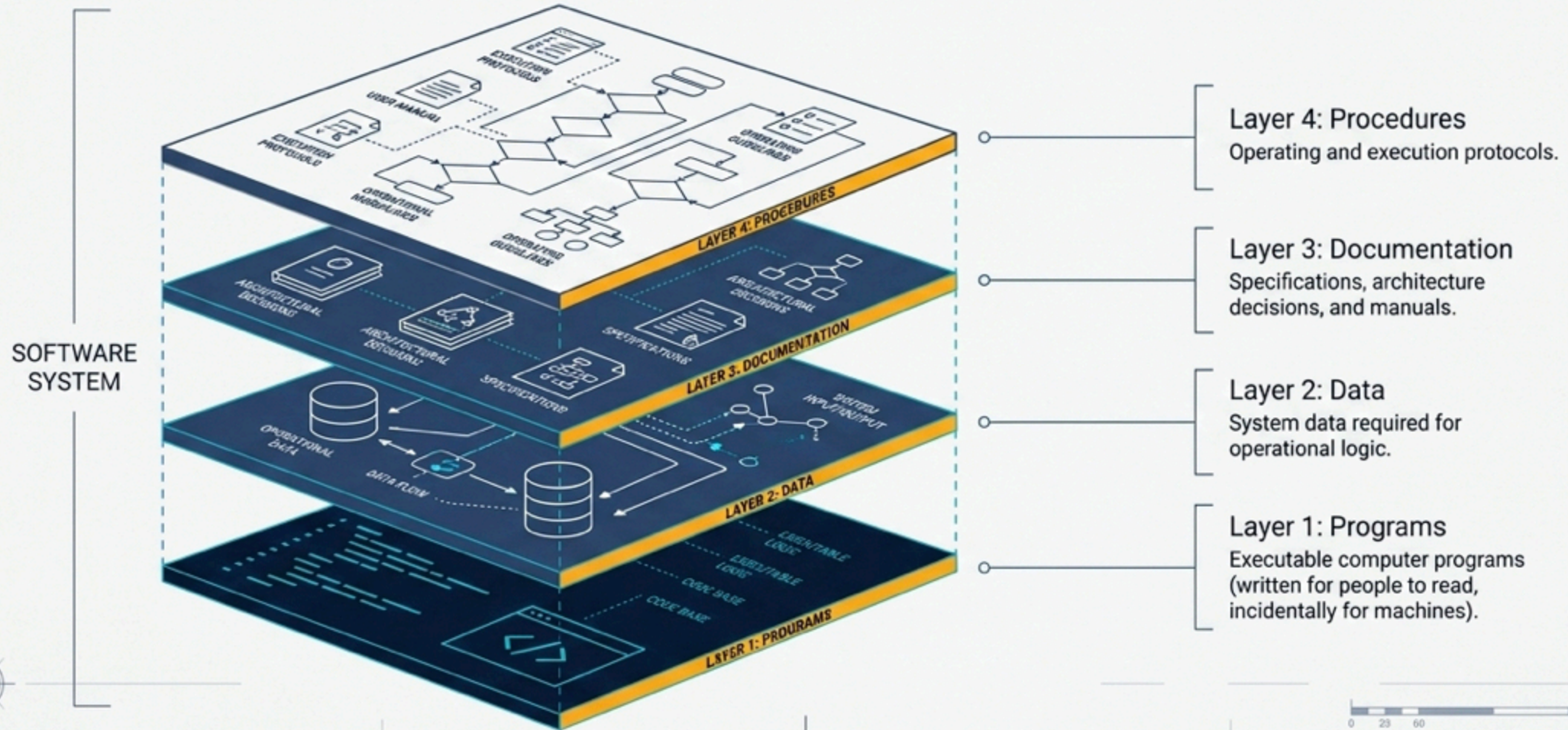
為何 1968 年的軟體危機無法僅透過採購運算速度更快的超級電腦或擴充硬體記憶體來解決？

- **A.** 危機是由 1960 年代後期的硬體製造延遲與矽晶片短缺所導致的。
- **B.** 危機是管理系統複雜度的認知與組織失敗，更快的硬體只會放大這個問題。
- **C.** 危機源於早期的程式語言缺乏基本的數學與計算功能。
- **D.** 危機發生是因為早期大型主機與現代的分散式雲端架構不相容。



The Anatomy of Software Exceeds Source Code

A failure in operational procedures is a failure of the software system. Physical hardware is explicitly excluded; software encompasses the intangible logic governing the system.



1.3 軟體冰山：超越原始碼的四大支柱

軟體 (IEEE 標準定義)： 與計算機系統運作相關的電腦程式、程序、以及可能伴隨的相關文件與資料。 „

- **1. 程式 (Programs)：** 可執行的機器碼與原始程式碼檔案（邏輯核心）。
- **2. 資料 (Data & Schemas)：** 資料庫綱要、設定檔、AI 模型權重（如 Knight Capital 設定檔出錯釀 4.4 億美元損失）。
- **3. 程序 (Procedures)：** 部署腳本、CI/CD 管線、災難備援 SOP（如 GitLab 刪庫且備份程序失效事件）。
- **4. 文檔 (Documentation)：** 架構設計 ADD、API OpenAPI 規格、使用者手冊（如 Therac-25 程式碼重用且無文件說明導致過載致死）。

1.3 軟體冰山陷阱

- 許多軟體專案之所以失敗，是因為開發人員與專案經理掉入了只關注軟體冰山頂端——**程式原始碼**的陷阱中。
- 他們僅以寫了多少行程式碼或交付了多少功能來衡量進度，卻忽略了資料庫遷移（資料）、CI/CD 部署管線與復原守則（程序）、以及系統設計與 API 規格（文檔）。
- 缺乏這四大支柱中的任何一項，系統就稱不上是「專業軟體」，而只是一個無法順暢部署、難以維護且運作危險的脆弱程式。

觀念檢驗：IEEE 軟體標準定義 (CCQ 3)

根據 IEEE 標準對軟體的正式定義，以下哪一項不被視為軟體的組成部分？

- **A.** 可執行的電腦程式與原始程式碼檔案。
- **B.** 系統資料庫綱要與組態設定檔案。
- **C.** CPU 處理器硬體與物理記憶體單元。
- **D.** 軟體的安裝說明與系統部署程序。



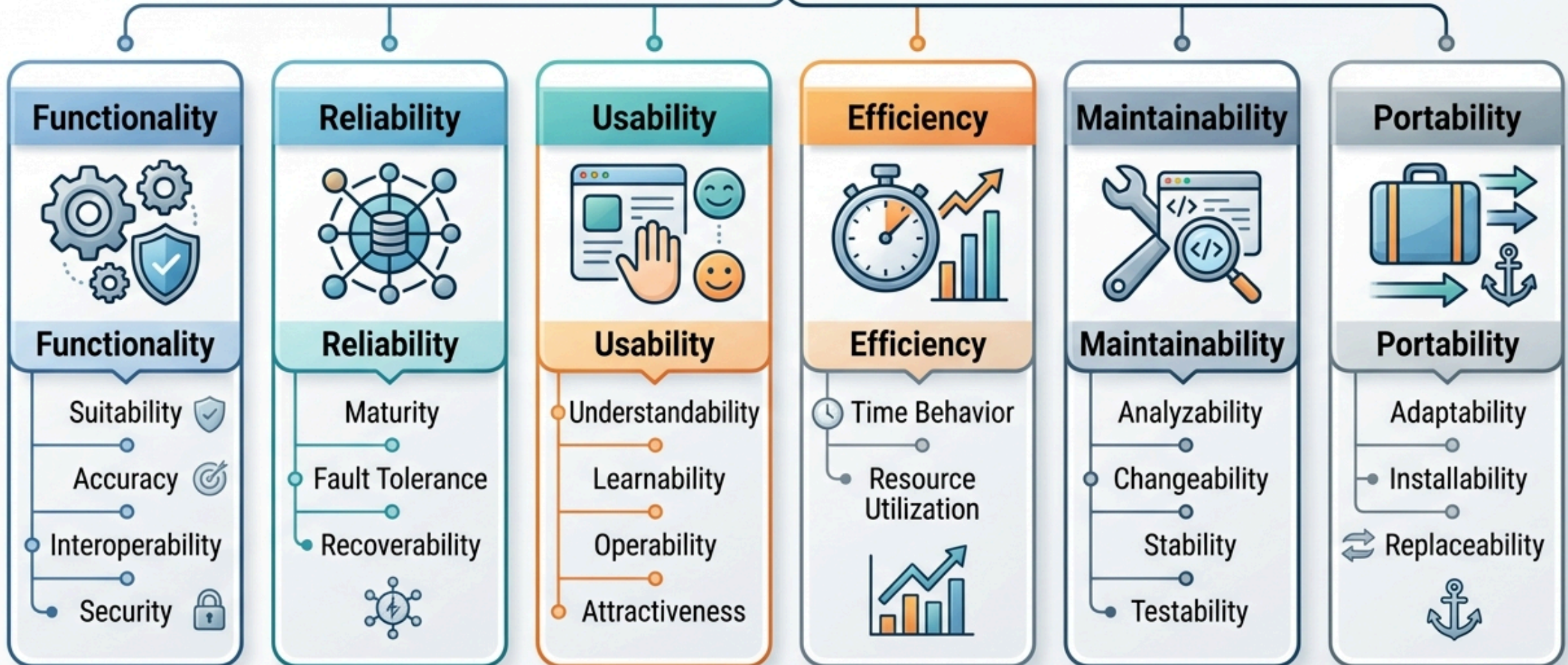
1.3 雙人分組活動：實務中的軟體冰山

雙人分組互動活動：

- 挑選一項熱門數位服務（如 **Google Maps**、**Uber** 或 **Spotify**）。
- 兩人合作針對四大支柱（程式、資料、程序、文檔）各列出一項具體產出物。
- 深入探討：如果一個工程團隊遺失了所有部署腳本與資料庫網要，他們能否單憑原始程式碼在產線上順利重建並維運業務？為什麼？



ISO/IEC 9126 Software Quality Model



1.4 ISO 9126 六大品質特徵

- **功能性 (Functionality)**：適用性、準確性、互操作性、安全保密性。
- **可靠性 (Reliability)**：成熟性、容錯性、易回復性。
- **易用性 (Usability)**：易理解性、易學習性、易操作性、美觀性。
- **效率性 (Efficiency)**：時間特性（回應時間、吞吐量）、資源利用性。
- **可維護性 (Maintainability)**：可分析性、可修改性、穩定性、可測試性。
- **可攜性 (Portability)**：可適應性、易安裝性、共存性、易替換性。

1.4 品質子屬性實務案例

- **容錯性 (Reliability)**：主要支付閘道故障時，系統自動捕捉異常並改走備用閘道重試，使用者無感。
- **易回復性 (Reliability)**：資料庫系統意外斷電重啟後，在 30 秒內透過預寫式日誌 (WAL) 自動恢復一致性。
- **時間特性 (Efficiency)**：搜尋 API 的回應延遲，在 99% 的情況下皆小於 80 毫秒 (p99 延遲 < 80ms)。
- **可測試性 (Maintainability)**：程式碼採用相依性注入設計，讓單元測試時能輕鬆 Mock 資料庫與外部 API。
- **易安裝性 (Portability)**：本地開發環境能透過 `docker compose up` 在 60 秒內自動建構完成。

觀念檢驗：實務問題與品質因素對應 (CCQ 5)

以下哪一個選項正確將真實世界的軟體問題與其對應的 ISO 9126 品質特徵進行了配對？

- **A.** 資料庫查詢需要 15 秒才能回傳結果 → Maintainability (Testability)
- **B.** 當第三方 API 斷線時，系統發生崩潰 → Reliability (Fault Tolerance)
- **C.** 由於程式碼緊密耦合，開發人員難以撰寫單元測試 → Portability (Adaptability)
- **D.** 應用程式無法在新版本的 macOS 上運行 → Usability (Operability)



1.4 課堂互動活動：品質權衡隨堂投票

品質權衡隨堂投票與小組討論：

- **系統 A：** 醫療型自動胰島素注射控制器
- **系統 B：** 爆紅的手機休閒小遊戲
- **投票：** 針對這兩類系統，挑選出最關鍵、不容妥協的 2 項 ISO 9126 品質特徵。
- **討論：** 為什麼在胰島素注射器中犧牲容錯性是致命的；而在休閒小遊戲中，卻能為搶快上線容忍非關鍵臭蟲？有哪些約束會損害品質？



YOUBIKE SMART BICYCLE SHARING SYSTEM DIAGRAM

YOUBIKE SMART BICYCLE



MOBILE APP

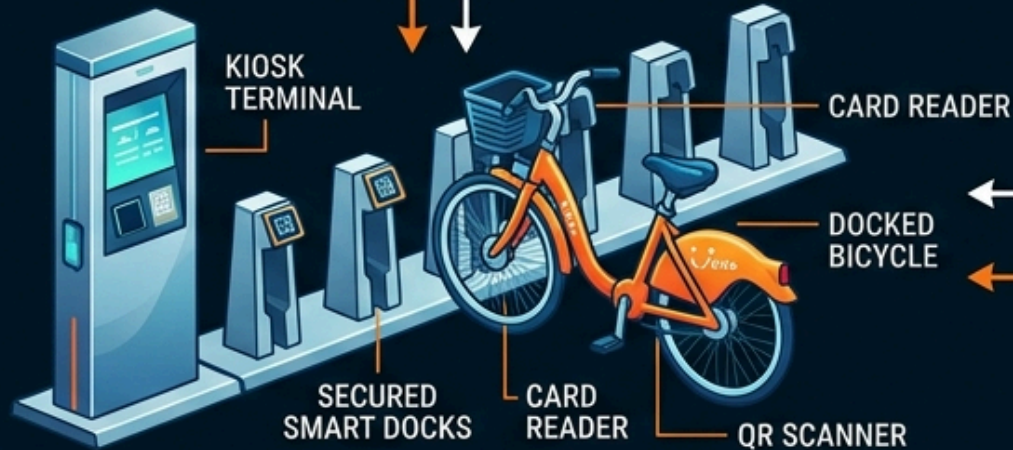


CELLULAR/IoT

**REAL-TIME LOCATION (GPS)
BICYCLE HEALTH
USAGE LOGS**

**BICYCLE
LOCK/UNLOCK**

**USER
RENTAL/RETURN**



API/MQTT

**STATION STATUS
RENTAL DATA
HEARTBEAT**

**USER AUTH
BOOKING
PAYMENT
UNLOCK COMMAND**



DOCKING STATION (DOCK)

BACKEND CLOUD SERVER

1.5 現代異質系統：以 YouBike 為例

- 現代的軟體系統很少是單一形態的程式，而是整合多領域的異質系統。
- **YouBike 智慧單車系統** 融合了多種軟體類別：
 - **嵌入式韌體**：自行車上的 IoT 車機、智慧鎖控制、智慧車柱（樁）。
 - **行動 App**：行動地圖查詢、車輛預約、掃碼扣款解鎖。
 - **網頁應用程式**：官方網站、會員帳戶管理、即時站點狀態查詢。
 - **企業後端系統**：資料庫交易處理、悠遊卡/信用卡支付後端、調度管理。
- 軟體工程的核心挑戰在於協調這些領域的軟體，平衡不同的約束與部署週期。

1.5 關鍵應用領域

- **Web & SaaS 平台**：強調高可用性、彈性伸縮、微服務與不停機部署（如 Netflix）。
- **行動 App**：考量電力消耗、觸控介面與不穩定行動網路（React Native/Flutter）。
- **企業系統 (ERP/CRM)**：重視 ACID 交易、複雜商務邏輯與資料治理（SAP）。
- **嵌入式韌體與 IoT**：即時性系統，記憶體資源受限，且不容許系統當機（醫材/車控）。
- **AI & 機器學習**：機器學習管線 (MLOps)、GPU 加速計算、向量資料庫與 LLM 推理。
- **科學與 CAD 應用**：高運算效能模擬套件，極度要求浮點數運算精確度。

1.5 課堂互動活動：系統分類

現代智慧系統的多元領域分類：

- 思考一台現代的 特斯拉智慧聯網電動車。
- 它涵蓋了本節所提到的哪些軟體領域？
 - 煞車與動力控制（即時嵌入式軟體）
 - 導航與車控螢幕（平板 UI 系統）
 - 車主手機搖控（行動 App）
 - 遙測數據傳輸與 OTA 系統（雲端 SaaS 與後端）
 - 自動輔助駕駛（邊緣 AI 模型）
- **討論：** 為何這些子系統的更新發布週期會有如此巨大的差異？



CODING



SOFTWARE ENGINEERING



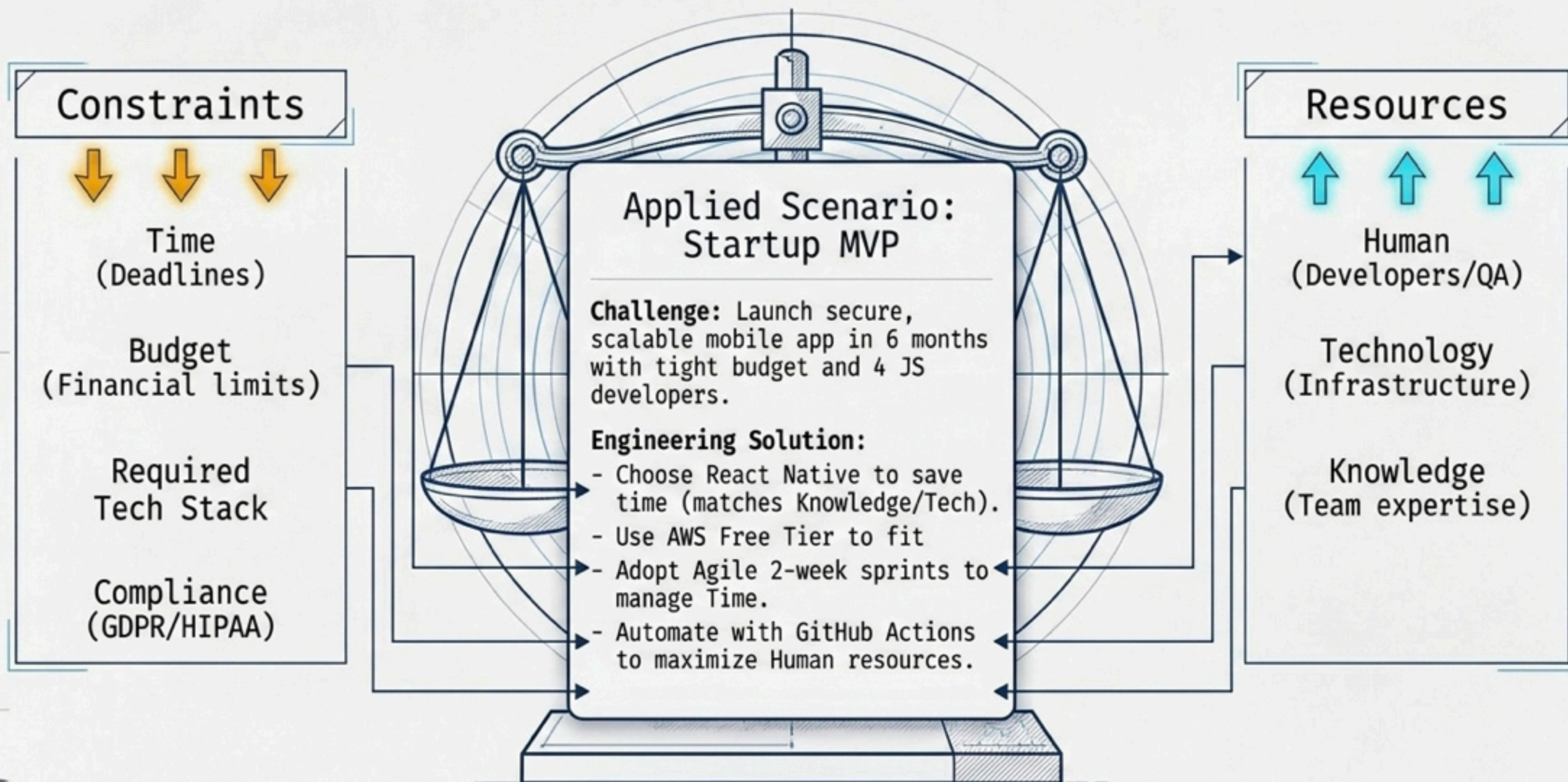
1.6 軟體工程的核心定義

軟體工程 (Software Engineering)：一門關於軟體生產所有階段的工程學科——從初始的需求規格書制定，直到系統上線後的維護與演進。

”

- **1. 工程學科 (Engineering Discipline)**：工程並非不計代價追求完美，而是要在有限資源與嚴格約束條件下妥協出合適解決方案的科學。
- **2. 軟體流程所有階段 (All Aspects of Production)**：軟體開發是由一整套系統化的軟體工程流程 (**SE Process**) 所引導，明確規範各活動、角色分工、產出物件與品質檢查點。

The Engineering Balancing Act



1.6 約束管理與資源優化

- **約束條件 (Constraints - 限制)：**

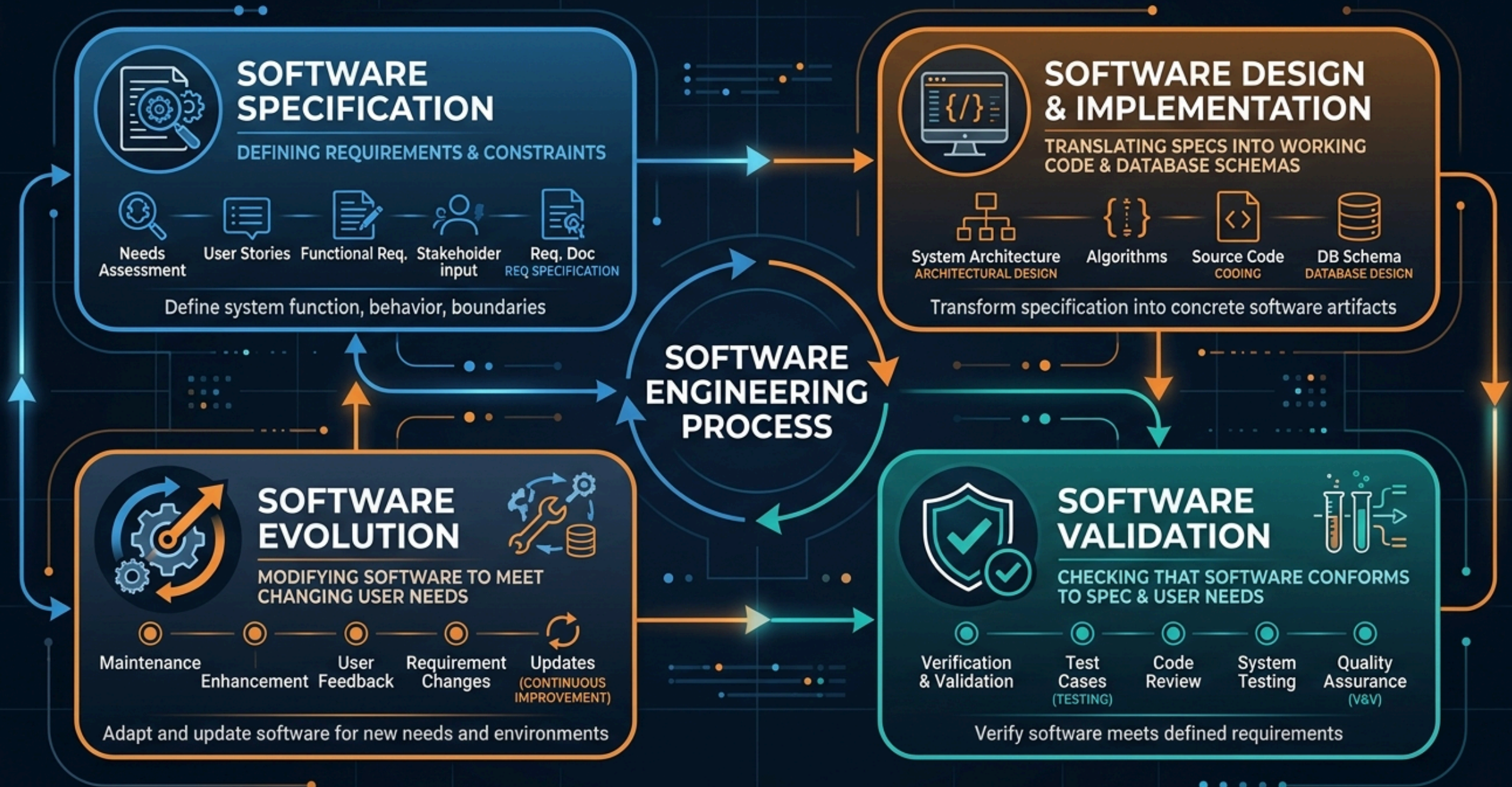
- **時間：** 專案交付死線、市場搶灘時間窗。
- **預算：** 開發人員薪資、雲端託管費、第三方套件授權費。
- **技術限制：** 舊系統相容性、硬體 CPU/記憶體限制。
- **法規標準：** GDPR 隱私法、HIPAA 醫療法規、PCI-DSS 金融支付標準。

- **資源資產 (Resources)：**

- **人力資源：** 開發人員技能、UI/UX 設計師、QA 團隊。
- **技術資產：** 雲端設施 (AWS)、CI/CD 自動化、成熟開源套件。
- **領域知識：** 對業務流程的理解與專案開發經驗。

1.6 案例研究：醫療新創公司 MVP 上線

- **挑戰：** 6 個月內推出一款病患醫生視訊 App，且預算極為吃緊，僅有 8 萬美元。
- **工程權衡方案：**
 1. **範疇優先級：** MVP 僅專注核心視訊與掛號；將保險給付延至 Phase 2。
 2. **跨平台開發：** 使用 Flutter/React Native 單一程式碼庫，節省 40% 前端人力。
 3. **託管式雲端：** 採用 HIPAA 合規的安全託管服務 (Firebase)，避免自建伺服器。
 4. **自動化 CI/CD：** GitHub Actions 自動測試，降低 QA 測試人力。
- **為什麼不選擇技術上「完美」的方案？**
 - 自建分散式微服務架構在技術上最完美，但因違反 8 萬美元與 6 個月的死線而被否決。
 - **滿意解 (Satisficing)：** 在所有約束內找出「足夠好」的妥協方案，這才是正確的工程決策。



1.6 軟體工程流程四大核心活動

活動	關鍵產出物	忽略後的嚴重後果
1. Specification (需求規格)	需求規格書 (SRS), 使用者故事	建構出完全錯誤的產品；面臨後期 100 倍修改成本
2. Design & Implementation	架構設計 ADD, 資料庫 ERD, API 規格	系統緊密耦合，產生大量技術債，架構崩塌
3. Validation (驗證與確認)	自動化測試套件, CI 測試報告	上線後頻繁崩潰、資料遺失或遭遇嚴重資安破壞
4. Evolution (維護與演進)	版本發布日誌, 資料庫遷移腳本	系統面臨軟體老化，技術債堆疊，最終報廢

觀念檢驗：工程動作與核心活動配對 (CCQ 6)

以下哪一個配對正確將特定的軟體工程動作與其對應的通用核心活動進行了對應？

- **A.** 進行利害關係人訪談以撰寫使用者故事 → Software Specification
- **B.** 撰寫自動化單元測試以模擬資料庫回應 → Software Design & Implementation
- **C.** 重構資料庫綱要以改善查詢速度 → Software Validation
- **D.** 將第三方的付款 API 替換為新的金流閘道 → Software Specification



1.6 課堂互動活動：案例情境分析

小組案例情境分析討論：

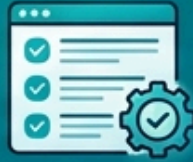
- **情境：** 某團隊花了 6 個月，為生鮮 App 開發了效能卓越、安全無虞的加密貨幣付款閘道。系統具備 100% 測試覆蓋率。然而上線後發現，社區長輩使用率為 0%，因為他們習慣只用現金與一般信用卡。
- **問題：** 這四大核心活動中，哪一個活動失效了？
- **關鍵教訓：** 為什麼 100% 的測試覆蓋率無法挽救本案例的失敗？
(驗證 Verification vs. 確認 Validation)



• CORE ELEMENTS OF SOFTWARE ENGINEERING •



Disciplines (Best Practices)



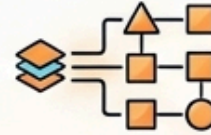
- Spec before design
- Design before code
- Interface-based
- Change management
- ADR (Architecture Decision Records)
- Code Reviews



Principles (Core Foundations)



- Abstraction
- Modularity
- Anticipation of Change
- Captration
- Open-Closed (OCP)
- KISS (Keep It Simple, Stupid)
- POLA (Principle of Least Astonishment)



Methods & Methodologies



- Waterfall
- Agile / Scrum
- Spiral Model
- TDD (Test-Driven Development)
- CI/CD & DevOps



Heuristics & Guidelines



- Nielsen's 10 UX Heuristics
- SOLID design
- Clean Code
- DRY (Don't Repeat Yourself)
- YAGNI (You Ain't Gonna Need It)

1.6 軟體工程的多元維度

- **專業紀律 (Disciplines):** 開發前設計規格、API 優先設計、落實 Code Review、撰寫架構決策 ADR。
- **基礎原則 (Foundational Principles):** 軟體理論根基：模組化、抽象化、預測變更、開閉原則 (OCP)。
- **流程方法論 (Process Methodologies):** 引導團隊運作框架：Waterfall、敏捷 (Agile/Scrum)、DevOps 自動化交付。
- **啟發式法則 (Heuristics):** 前人實務經驗：SOLID 設計原則、Clean Code 乾淨程式碼、DRY、YAGNI 原則。

1.6 破解常見的軟體迷思

- 迷思 1：「專案進度落後了，招募新工程師進來就能解決。」
 - 現實 (Brooks's Law)：在落後的專案中加入人手，只會讓專案更落後（新人培訓成本與 $O(n^2)$ 溝通管道暴增）。
- 迷思 2：「軟體具有高度彈性，在後期修改需求很便宜。」
 - 現實：後期修改會迫使已完成的資料庫、API 與代碼打掉重練，修改成本常是前期的 100 倍。
- 迷思 3：「只要把程式外包，我們就不需要懂技術管理。」
 - 現實：外包專案若缺乏技術管理、持續整合與嚴格架構監督，最終交付的系統往往是一場災難。

Myth: Changing requirements late is cheap!

Requirement change early during blueprint design (Cost: 1x)



Requirement change at the very end of construction (Cost: 100x)



觀念檢驗：布魯克斯法則 (CCQ 7)

專案目前落後進度 3 週，距離預定上線日僅剩 2 週。專案經理決定緊急招募 4 名新工程師來加速開發。根據布魯克斯法則 (Brooks's Law)，最可能發生的結果為何？

- **A.** 專案將會提早 1 週順利上線。
- **B.** 專案將會面臨更嚴重的延遲，因為資深開發人員必須停下工作來培訓與協調新進人員。
- **C.** 新進人員的加入對專案時程完全沒有任何影響。



1.7 現代工程自動化工具鏈

- **Version Control (Git)**：分支管理策略，協作程式碼審查。
- **現代 IDE (如 VS Code)**：即時語法錯誤檢查、重構輔助與除錯環境。
- **CI/CD 自動化管線**：自動編譯、單元測試、弱點掃描並容器化部署 (GitHub Actions)。
- **自動化測試**：單元測試 (JUnit、PyTest)、整合測試與 E2E 測試。
- **可觀測性與 APM**：OpenTelemetry 鏈路追蹤、日誌整合與 Sentry APM。
- **容器化編排**：隔離應用依賴 (Docker 容器) 與伺服器叢集編排 (Kubernetes)。
- **專案追蹤管理**：Scrum 看板、缺陷與功能待辦追蹤 (Jira、Linear)。
- **靜態代碼分析**：不執行代碼下，掃描原始碼中的潛在臭蟲與 code smell (SonarQube)。
- **API 開發協作**：API 設計、Mock、測試與共享規格 (Postman)。
- **基礎設施即代碼 (IaC)**：透過版本控制的設定檔宣告雲端資源 (Terraform)。

1.7 AI 在生命週期各階段的效益與風險

生命週期階段	AI 帶來的效益	AI 伴隨的風險與隱憂
1. 需求與分析	快速草擬 User Stories、提煉驗收條件	幻覺不實需求與約束、遺漏隱性組織知識
2. 架構與設計	推薦合適架構模式、自動產生資料庫 ERD	過度設計、忽略系統實體延遲與安全隔離阻斷
3. 實現與編碼	自動補齊程式碼、加速樣板程式代碼產生	「Vibe Coding」 陷阱、安全漏洞、授權疑慮
4. 測試與 QA	自動生成單元測試案例、生成邊界測試數據	「迴音室測試」（只在驗證 AI 出錯的代碼）
5. 演進與維護	自動解讀遺留代碼、自動草擬 API 文檔	自動重構時引入隱性迴歸臭蟲 (Regression Bugs)

JUST COMPILE IT.
TRUST THE VIBES!

CRITICAL
SYSTEM FAILURE

PRODUCTION SERVERS

LOGS: ERROR 500

LOGS: ERROR 500

CRITICAL OVERLOAD!

KERNEL PANIC!

FIRE IN DATA CENTER!

BREAK GLASS
IN CASE OF
VIBE CHECK

CODE
&
VIBES

1.7 隨堂討論：AI 的「Vibe Coding」工程挑戰

「Vibe Coding」的現場投票與討論：

- **投票：** 當你使用 AI 程式助理（如 Copilot 或 ChatGPT）時，你在按下 Commit 提交前，有多少比例會逐行閱讀並完全理解它所產生的每一行程式碼？
(每次都逐行看懂 / 大部分時候會看 / 很少看 / 從來不看)
- **討論：** 假設 AI 幫你寫了一段高複雜度的非同步資料庫處理邏輯，且順利通過了本地端 2 項單元測試，但你其實看不懂其內部原理。在軟體工程紀律下，你是否應該直接將其 Merge 到正式分支？你該採取哪些步驟來驗證其安全？





1. PUBLIC

Act consistently with the public interest.
Prioritize safety, health, and welfare.



2. CLIENT & EMPLOYER

Act in the best interest of the client and employer.
Maintain confidentiality.



3. PRODUCT

Ensure products meet high standards.
Aim for quality, safety, and reliability.



4. JUDGMENT

Maintain integrity and independence.
Exercise sound professional judgment.



5. MANAGEMENT

Use fair and effective management practices.
Support ethical work environments.



6. PROFESSION

Advance the integrity and reputation of the profession.
Report ethical violations.



7. COLLEAGUES

Be fair to and supportive of colleagues.
Cooperate for shared success.



8. SELF

Participate in lifelong learning.
Improve knowledge and skills continuously.

1.8 ACM/IEEE 倫理守則 8 大核心原則

1. **Public (公眾)**：以公眾利益為最高指導原則，優先保障安全、健康與福祉。
2. **Client & Employer**：在符合公眾利益的前提下，維護客戶與雇主的利益。
3. **Product (產品)**：確保交付的產品符合專業最高可靠度與安全標準。
4. **Judgment (判斷)**：保持專業評估的獨立自主性，拒絕對不實技術宣稱簽署背書。
5. **Management (管理)**：推動合乎道德的管理，不給予不實估算或強加不合理壓縮。
6. **Profession (專業)**：藉由同儕審查與分享知識，維護軟體工程的誠信與聲譽。
7. **Colleagues (同事)**：待同事公平客氣，營造互助與正面建設性回饋的氛圍。
8. **Self (自身)**：終身學習，精進技術，落實道德的開發實踐。

1.8 當工程倫理失效時：真實世界的案例

- 福斯汽車「柴油門」事件 (2015)：

- 工程師受命編寫軟體偵測車輛是否在實驗室進行排氣檢測，以暫時降低 NO_x 排放；實際道路排放高達法定 40 倍。導致數百億罰款與嚴重污染。

- 劍橋分析數據醜聞 (2018)：

- 工程人員協助不正當收集與濫用數千萬社交使用者的數據以操弄政治選舉。

- 計劃性報廢 (Planned Obsolescence)：

- 在系統更新中加入限速機制，刻意拖慢舊硬體效能以逼迫消費者換新機。

DECEPTIVE UI/UX DARK PATTERNS



1.8 UI/UX 中的欺騙性「黑暗模式」

- **1. 蟑螂屋 (Roach Motel) :**
 - 訂閱只需一鍵完成；取消訂閱卻需要打電話給客服或在隱密選單迷宮中摸索。
- **2. 誘導式確認 (Confirmshaming) :**
 - 在拒絕按鈕上加上帶有道德羞辱性或愧疚感的字眼（如*「不，謝謝，我討厭省錢」*）。
- **3. 悄悄塞入購物車 (Sneak into Basket) :**
 - 在最後結帳點自動預先勾選加購險、加購商品，利用粗心誘導使用者付錢。
- **4. 虛假緊急 (Fake Urgency) :**
 - 製造虛假的搶購人數（「34 人正在看此房」）或虛假的特價倒數計時器逼迫下單。

觀念檢驗：工程倫理守則 (CCQ 5 - 續)

** under the ACM/IEEE Code of Ethics, if an employer directs an engineer to implement an algorithm that falsifies safety compliance reports, what is the engineer's obligation?**

- **A.** 順從雇主，因為雇主支付工程師的薪水。
- **B.** 拒絕並向上陳報，因為公眾利益的優先權高於對雇主的忠誠。
- **C.** 實作程式碼，但不要寫任何系統文件。
- **D.** 將此程式碼外包給外部廠商開發以規避責任。



1.8 課堂活動：黑暗模式偵探

黑暗模式偵探活動：

- **找尋：** 回想你最近在使用哪些 App 或網站時，曾遇過哪種類型的黑暗模式？
- **分析：** 這項惡意設計違反了 ACM/IEEE 倫理守則中的哪一條原則？
- **重新設計：** 如果由你主導，你會如何重新設計該操作流程，使其既符合商業轉換率，又尊重使用者的自主權與知情權？



1.9 軟體工程常見問答 (FAQ)

- **Q1：寫程式與軟體工程有何不同？**

- *回答：* 寫程式是撰寫原始碼。軟體工程是融入時間軸的程式開發，包含多人協作、在時間與預算等限制下折衷出「滿意解」、維護品質特徵與規畫長期演進。

- **Q2：為何程式正確且測試 coverage 100% 仍會失敗？**

- *回答：* 軟體有四大支柱。資料出錯、維運程序出包、或文檔缺失皆會失敗。且若需求規格 (Specification) 階段錯誤，就只會「正確地蓋出沒人要的產品」。

- **Q3：滿意解 (Satisficing) 與最優解 (Optimized) 有何差別？**

- *回答：* 受限於預算與時程約束，技術上完美的「最優解」往往不可行。工程的核心在於找出符合所有限制且「足夠好」的「滿意解」。

- **Q4：AI 的 Vibe Coding 風險與防範？**

- *回答：* 憑感覺盲目使用 AI 程式碼會引入安全性漏洞。防範之道在於落實 Code Review、寫程式前先寫測試案例，並將 AI 代碼視為需專業工程審查的初稿。

課堂總複習：隨堂填空挑戰

檢測你對本章基礎概念的掌握度：

1. 根據 IEEE 正式定義，軟體由程式、資料、程序以及【 _____ 】共同組成。
2. 指出「在已經落後的專案中招募人手，只會讓專案更落後」的法則是【 _____ 】法則。
3. 【 _____ 】品質模型定義了包含功能性、可靠性與可維護性在內的 6 大軟體品質特徵。
4. ACM/IEEE 軟體工程倫理守則的第一條原則，要求將【 _____ 】利益放在第一位。



參考文獻與延伸閱讀

- Sommerville, Ian. *Software Engineering* (10th Edition). Pearson. [Official Website](#)
- Brooks, Frederick P. *The Mythical Man-Month: Essays on Software Engineering*. Addison-Wesley.
- ISO/IEC 9126-1:2001. *Software engineering — Product quality — Part 1: Quality model*.
- ACM/IEEE Joint Task Force on Software Engineering Ethics. *Software Engineering Code of Ethics*. [IEEE CS](#)
- Brignull, Harry. *Deceptive Patterns: Exposing the Tricks Tech Companies Use to Control You*.